A Food Waste Rescue Marketplace and Demand Forecasting Application

Eddie Allman, Marcos Ashton, Theo Chan, Rafe Crowley, Caspar Hadley, Ben Parker, Charlie Winders, Jason Yeung

1. Executive Summary

1.1 Problem Identified

Using data from 2021 and 2022, it has been estimated that 11% of the UK's total food waste has been created by the hospitality and food service industry[3]. Globally, food loss and waste accounts for 8–10% of greenhouse gas emissions[1], meaning that hospitality food waste is directly responsible for up to 1.1% of total greenhouse pollution. Shifting focus to employee statistics, a survey on staff productivity has shown that over an average 8 hour working day, only 2 hours and 53 minutes are spent productively[2]. With limited productive time and long working hours, many employees resort to ordering takeaway meals at the end of the day, adding unnecessary personal expense while perfectly edible restaurant food goes to waste.

For food service operators, overproduction costs money. Disposal fees, spoilage, and over-purchasing all eat into margins that are already thin. Organisations benefit too – end-of-day food bundles are cheaper than a Deliveroo order, and offering them as a staff perk costs nothing to run. Byte Me targets both sides at once, which is what makes the B2B model more compelling than another consumer-facing food app.

1.2 Proposed Solution

Byte Me is a business-to-business food rescue marketplace that connects food service sellers with organisations looking to provide affordable meals for their employees. Restaurants and cafés list surplus food bundles at reduced prices before the end of their trading day. Organisations can then browse, reserve, and collect these bundles using a secure claim code system. The platform includes a demand forecasting engine that helps sellers predict reservation volumes and reduce overproduction. A gamification layer with rescue streaks and achievement badges incentivises consistent participation, while seller analytics dashboards provide insight into sell-through rates, waste avoided, and carbon impact. By targeting the commercial sector rather than individual consumers, Byte Me enables bulk food rescue at scale, reducing hospitality food waste while offering organisations a practical employee benefit.

1.3 Sprint 2 Deliverables

Sprint 2 took the system from a working prototype to something we'd actually hand over to a client. Analytics went from two basic views to six. Forecasting went from one EMA model to a three-model comparison with MAE, RMSE, and Brier Score on held-out data. We built issue reporting end-to-end across six API endpoints with a full OPEN → RESPONDED → RESOLVED workflow. Testing ended up at 28 backend test files (142 methods) and 13 frontend files (220 tests), all on two GitHub Actions CI pipelines. A four-document client handover pack was produced for deployment continuity.

1.4 Deployment Status

The system is live on Railway. All 41 API endpoints are up, Flyway migrations have been applied, and the seeded dataset is in place. Setup scripts for macOS and Windows are included so a new maintainer can get the system running locally without manual configuration.

1.5 Key Risks and Mitigations

The main reliability risk was overselling under concurrent load. We pushed the capacity check to the database layer – a PostgreSQL trigger (`reservation_capacity_on_insert`) and a `chk_qty_reserved_le_t` constraint – so it can't be bypassed by the application. BCrypt handles password hashing and JWTs expire after 24 hours; the lack of a token refresh mechanism is a known gap. The forecasting weather input is a boolean flag rather than a live API feed, which limits accuracy but removes an external service dependency.

2. Requirements Traceability

Features included in the final version of Byte Me, with delivery status against the original MoSCoW plan.

2.1 Prioritisation Rationale

The prioritisation strategy focuses on delivering a reliable vertical slice in Sprint 1 that satisfies all mandatory coursework requirements while clearly demonstrating system value. Core marketplace functionality (posting, reserving, claim code verification, and streak updates), baseline analytics, forecasting, and a fully seeded dataset are treated as non-negotiable because they directly map to the marking criteria and defined measures of success (zero oversell rate, functioning forecasting evaluation, and measurable waste impact). Sprint 2 expands depth rather than breadth, strengthening analytics, forecasting accuracy, governance documentation, and maintainability to support client handover and professional practice evidence. Enhancements such as maps, reviews, and extended gamification are deliberately deprioritised to avoid scope creep, while payment processing and gambling-style mechanics are explicitly excluded due to specification constraints and legal risk. This structured approach ensures technical completeness, risk control, and alignment with both CW1 and CW2 assessment objectives.

2.2 MoSCoW Prioritisation with Delivery Status

Must Have (Sprint 1)	**Should Have (Sprint 2)**	**Could Have (If Time)**	**Will Not Have (Out of Scope)**
Login and role-based access control *Delivered*	UX polish and accessibility refinement *Delivered*	Advanced dietary preference filters *Not delivered*	In-app payment processing (explicitly disallowed)
Clean landing page *Delivered*	Map view of nearby stores *Not delivered – see note*	Seller and buyer review system *Not delivered*	Paid spin / gambling-style rewards
Seller bundle posting (quantity, price, pickup window, allergens) *Delivered*	Issue reporting workflow *Delivered*	Google Maps walking-time integration *Not delivered*	Real-time delivery tracking
Consumer browsing and reservation system *Delivered*	Terms of Service page *Delivered*	"Meet the Developers" page *Not delivered*	Live chat support
Claim code generation and verification *Delivered*	Cookies policy *Delivered*	Extended gamification features *Not delivered*	Full e-commerce checkout system
Rescue streak tracking (mandatory requirement) *Delivered*	Data protection and privacy policy *Delivered*	Advanced seller dashboard customisation *Not delivered*	Social media integration
Minimum seller analytics (sell-through and waste proxy) *Delivered*	Full analytics suite (pricing and operational insights) *Delivered*	Leaderboard functionality *Not delivered*	Complex loyalty marketplace
Baseline forecasting model and evaluation snapshot *Delivered*	Improved forecasting model with recommendation logic *Delivered*	Weather API integration *Not delivered*	AI chatbot assistant
Seeded dataset (25+ sellers, 250+ bundles, 400+ reservations) *Delivered*	Maintainer/admin tools *Delivered*	Extended accessibility enhancements *Not delivered*	Real payment gateway integration
Allergen display and safety disclaimer *Delivered*	Formal accessibility audit *Partially delivered – see note*		

Table 2.1: MoSCoW prioritisation with final delivery status.

2.2.1 Scope Decisions

All Must Have items were delivered in Sprint 1. Most Should Have items were delivered in Sprint 2. Two need explaining.

The map view wasn't delivered. It would have required a third-party mapping API, key management, and geolocation permissions in the browser – meaningful effort for something that didn't map to any core marking criterion. We put that time into forecasting depth and analytics instead.

The accessibility audit was partially delivered. We made real improvements (read-aloud, contrast fixes, keyboard navigation) and ran a self-evaluation against WCAG 2.2 AA, getting partial conformance. No Lighthouse or axe tooling was used and no screen reader was built, so the audit can't be considered fully independent.

Could Have features weren't delivered – that was the right call. They don't affect reliability, forecasting, or any marking criterion, and chasing them would have spread Sprint 2 too thin.

2.2.2 Sprint Allocation and Justification

Feature	**Sprint**	**Justification**
End-to-end marketplace workflow (Post → Reserve → Claim → Streak update)	Sprint 1	Required vertical slice for CW1 demonstration. Core system value depends on reliable end-to-end execution.
Seller analytics (minimum two views)	Sprint 1	Explicit CW1 requirement demonstrating measurable waste reduction and operational insight.
Baseline forecasting model	Sprint 1	Specification requires at least one baseline forecast and evaluation snapshot in CW1.
Seeded dataset	Sprint 1	Mandatory for marking; system must operate without external services.
Issue reporting system	Sprint 2	Required for full v1.0 system but not part of Sprint 1 vertical slice.
Expanded analytics suite	Sprint 2	CW2 requires deeper analytical evaluation and pricing effectiveness insights.
Improved forecasting with recommendation logic	Sprint 2	CW2 requires comparison against baselines on held-out data and actionable recommendations with confidence labels.
Maintainer tools and deployment documentation	Sprint 2	Part of the client handover pack and professional practice evidence for final submission.
Accessibility refinement	Sprint 2	Baseline compliance achieved in Sprint 1; full audit and refinement in Sprint 2.
Policies (ToS, Privacy, Cookies)	Sprint 1/2	Basic policy links included in Sprint 1; expanded documentation aligned with final system in Sprint 2.
Maps, reviews, extended gamification	If time (Sprint 2+)	Enhancement features that do not affect system reliability, forecasting accuracy, or core marking criteria.
Payments and gambling-style mechanics	Out of Scope	Explicitly prohibited by coursework brief and introduce unnecessary legal and security risk.

Table 2.2: Sprint allocation and rationale aligned to CW1 and CW2 marking criteria.

3. Architecture v2

Diagrams and explanations from the design and implementation phases of our agile development. Included is information about our frontend, backend, and database systems, along with data flow and user interaction specifics. This chapter reflects the final delivered system, including structural changes made during Sprint 2 and a discussion of the architectural trade-offs.

3.1 Sitemap and User Journey

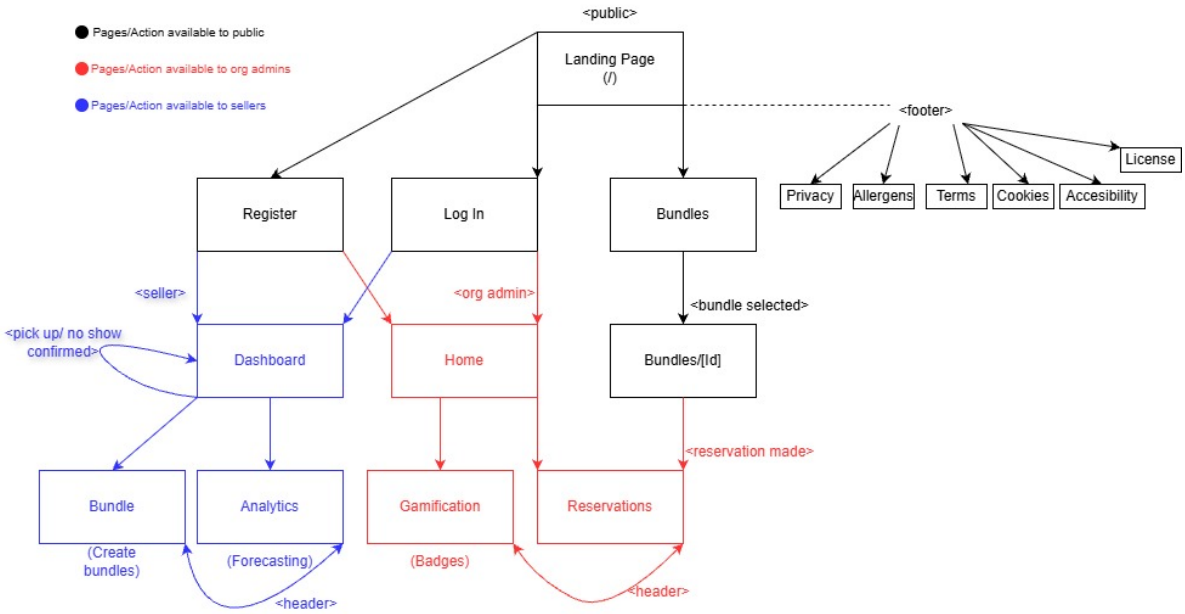


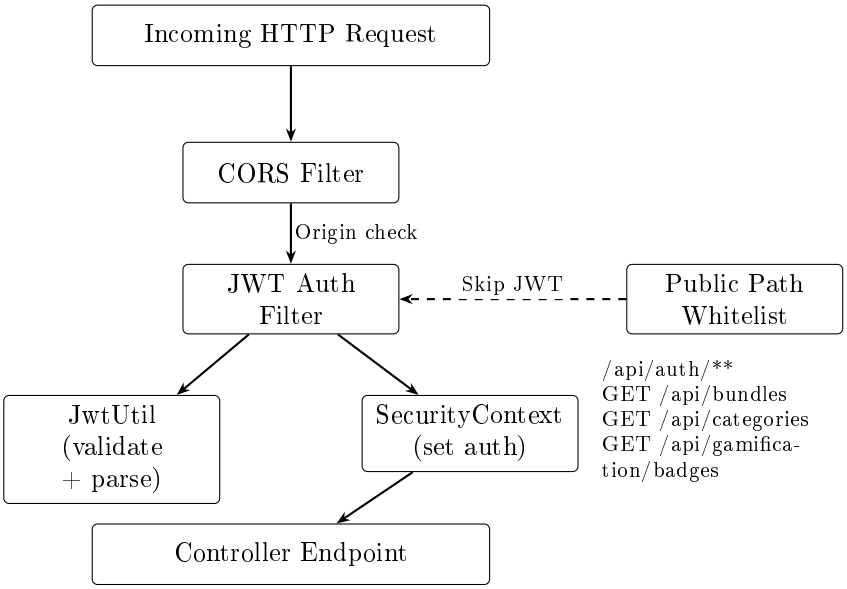
Figure 3.1: User Flow Diagram for Byte Me

3.1.1 User Flow Justification

Public -> When a user initially loads the website they see our landing page. This displays key details about how the project works and its goals, as well as giving them access to log in and register pages, alongside key pages via. links in the header and footer. Without being logged in, a user can access all legality associated pages through the footer, specifically; the privacy policy, terms and conditions, cookies policy, allergens notice, accessibility statement, and license pages. It is important these remain publicly available for full disclosure to users even before they have made or logged into an account. All users are also given access to the bundles page, and through this the bundles/[id] page, giving them key details, dates and price points of any available or soon to be available bundles from our sellers.
Organisation Admins -> Once an org admin has logged in or registered via. the pages on our header, they will gain authorisation to all org admin specific pages. For now, this is the gamification page, and the reservations page. They will also gain access to new features within the bundles/[id] page allowing them to reserve bundles under their organisation name and acquire a claim code for picking up said bundle. Any reservations made can be re accessed by visiting the reservations page which is made available to org admins in the header. This allows users to reserve bundles in advance and return later to cancel or retrieve their claim code, giving our users more freedom and ease when using our service. The gamification page displays any badges achieved and any progress towards over achievable badges. This allows companies to see the

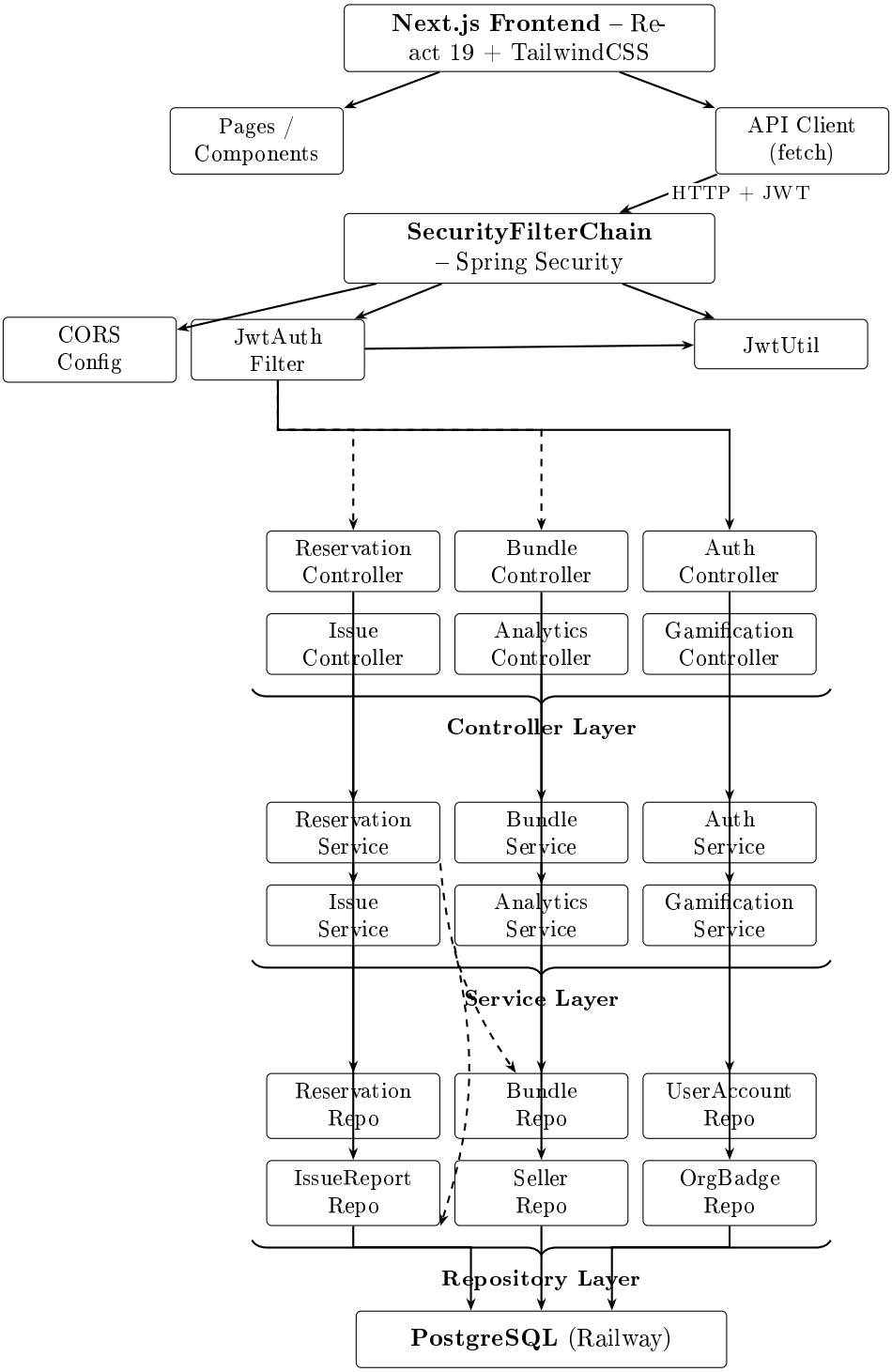
important steps they are making to help the environment and reduce food waste, while giving them real, certified badges they can display to show the work they are doing to others.
Sellers -> Once logged in as a seller, the user is redirected to their personalised seller dashboard page. This gives them insight into how many bundles have been posted, sold and cancelled, alongside their active bundles and links to seller specific pages. Sellers can access a special bundle page, providing a form for them to create and post bundles which are made available immediately or at a scheduled time to all users on our platform. We believe it is important for our sellers to have a simple UI for bundle creation as we understand this is not the main focus of their business, and the ease of creation increases the amount of bundles made, and therefore reducing food wasted. Sellers can also access the analytics page, giving detailed forecasting of their demand in general, and for individual bundles. This helps them tweak supply to further minimise food waste, as well as saving money for their businesses, providing a real incentive for our sellers to waste as little food as possible.

3.2 Security Integration



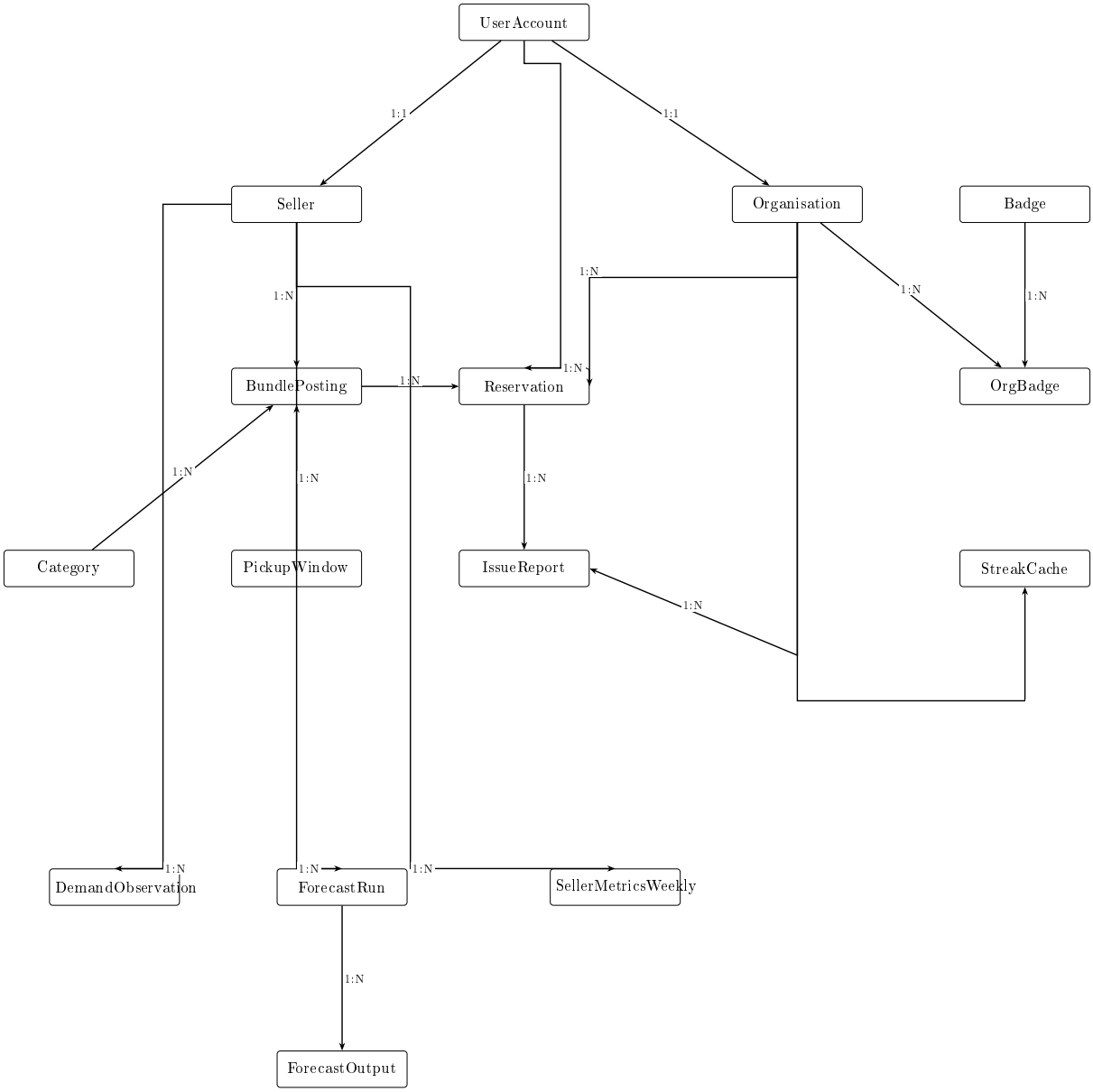
CORS runs first. JWT filter validates tokens for non-whitelisted paths and sets the SecurityContext. Sessions are stateless; passwords use BCrypt.

3.3 Full-Stack Integration Diagram



Frontend calls the backend via REST with JWT. Each controller delegates to its service, which queries repositories. Dashed arrows show cross-service repo access.

3.4 Entity Relationship Integration



UserAccount maps 1:1 to Seller or Organisation by role. Sellers create bundles; organisations reserve and file issues. OrganisationBadge is a many-to-many join. Category and PickupWindow are lookup tables. The Sprint 2 V2 migration added `DemandObservation`, `ForecastRun`, `ForecastOutput`, `SellerMetricsWeekly`, and the `forecast_action` tracking table.

3.5 Backend Class Breakdown

The backend for this project uses Java as its main language as it was a language we were all familiar with that required enough structure to force an organised workflow amongst the developers. We used the spring framework to manage transactions between the backend and database, making the process easily standardised.

The back end is comprised of three components:

- **Container Objects** – Each object represents a row in a database table, with attributes that store each value along the row.
- **Repositories** – Methods contain SQL queries that are run on the database. The resultant tuples are converted into container objects.

- **Controllers** – Interface with the frontend, providing functions that run the methods in repositories to retrieve requested data and send up to the frontend. In some cases such as forecasting, the data is operated on before results like predictions are sent instead.

These three work together to facilitate data flow between the frontend and database.

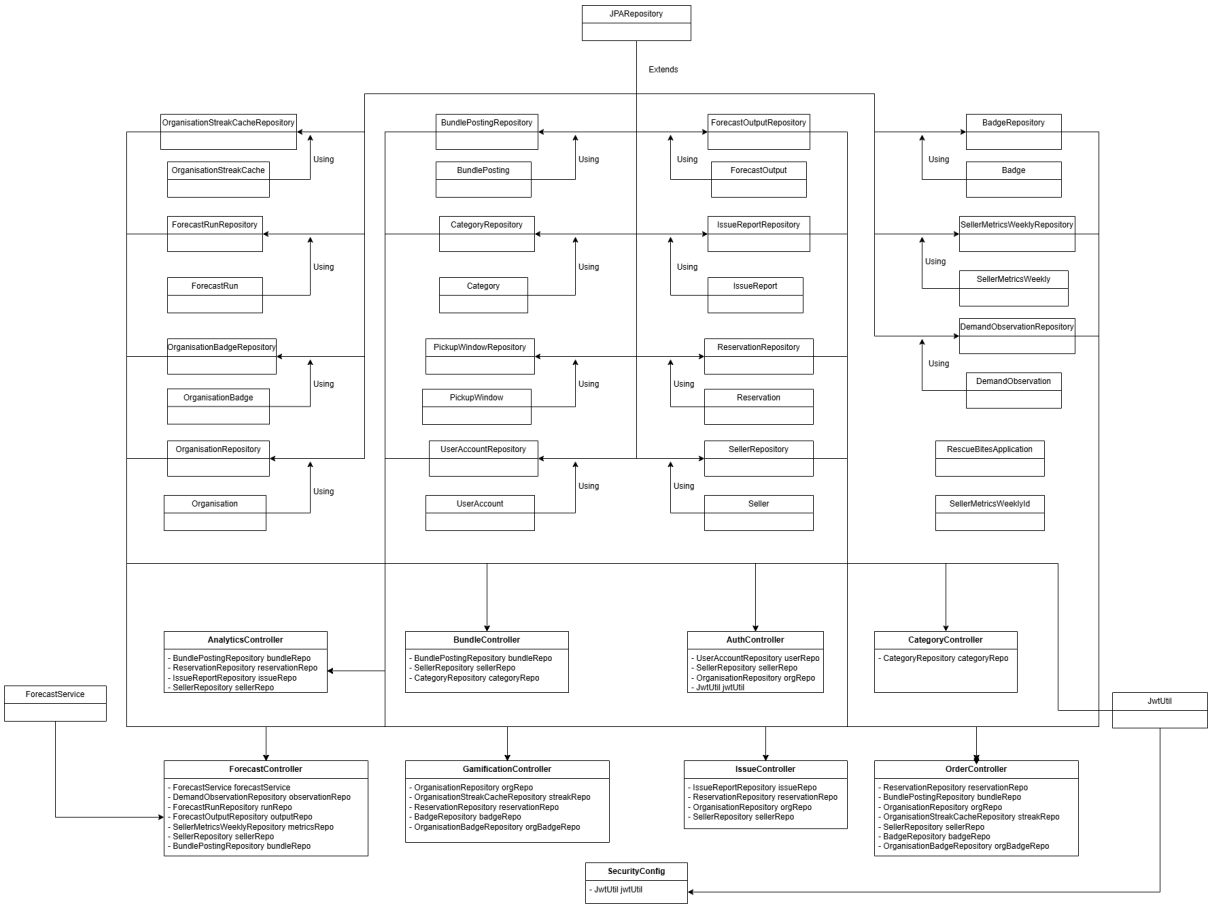


Figure 3.2: Backend Class UML Diagram

3.6 Database Architecture

3.6.1 Deployment Architecture

The system is deployed on Railway, which provides a managed PostgreSQL database and application hosting environment. The system follows a standard three-layer architecture:

- **Frontend:** Web interface for sellers and organisations.
- **Backend service:** Handles authentication, role-based access control, reservation logic, and workflow transitions.
- **Managed PostgreSQL:** Persistent data storage and enforcement of critical business rules.

All secrets (database URLs, credentials) are configured via environment variables within Railway and are not stored in the repository. The backend is stateless, allowing Railway to manage deployment and scaling without affecting application logic.

The database is intentionally responsible for enforcing key invariants, including oversell prevention, role validation, and rescue-event integrity. This design ensures correctness even if the application layer encounters unexpected input.

3.6.2 Data Protection and Cloud Infrastructure

The system is deployed on Railway, which runs on AWS-managed cloud infrastructure. This provides a secure, production-grade environment for both the backend service and the PostgreSQL database.

Using AWS-backed infrastructure offers several data protection advantages:

- **Managed database hosting:** PostgreSQL is hosted in a managed environment with secure configuration, automated backups, and infrastructure-level isolation.
- **Encrypted connections:** Database connections use encrypted transport (TLS), protecting data in transit.
- **Infrastructure-level security controls:** AWS data centres provide physical security, network segmentation, and resilience measures that would not be achievable in a local deployment.
- **Environment-based secrets management:** Sensitive credentials (e.g., database URLs) are stored as Railway environment variables and are not committed to the repository.

At the application level, additional data protection measures include password hashing, role-based access control, and strict database constraints to prevent unauthorised or inconsistent data states.

The system does not process payment details or highly sensitive personal data, but deploying on AWS-backed infrastructure ensures that user credentials and operational data are handled within an industry-standard cloud security environment. This supports responsible data management and reduces operational risk compared to local or unmanaged hosting.

3.6.3 Data Model Design

The schema is organised around the real-world flow of surplus food:

1. **User and Role Management** – `UserAccount`, `Seller`, and `Organisation` define actors and enforce role constraints.
2. **Marketplace Core** – `BundlePosting` and `Reservation` implement the posting and reservation workflow.
3. **Impact and Gamification** – `OrganisationStreakCache`, `Badge`, and `OrganisationBadge` support rescue streak tracking.
4. **Issue Handling** – `IssueReport` enables structured feedback and resolution.
5. **Forecasting and Analytics** – `DemandObservation`, `ForecastRun`, `ForecastOutput`, and `SellerMetricsWeekly` store demand history, model evaluations, and seller insight metrics.

3.6.4 Core Workflow Implemented

The final system supports:

1. Sellers create bundles with quantity limits and pickup windows.
2. Organisations reserve bundles and receive a hashed claim code.
3. Sellers verify pickup and mark reservations as collected.
4. A rescue event records estimated meals and CO_2e saved.

5. Organisation streak data updates accordingly.

Capacity checks are enforced transactionally in the database, guaranteeing an oversell rate of zero. Status transitions are recorded to preserve traceability and support evaluation.

3.6.5 Explanation

At a simple level, the system allows a seller to list surplus food and an organisation to reserve it. The database guarantees that no more bundles can be reserved than are available. When a bundle is collected, the system records the environmental impact and updates the organisation's rescue streak.

Railway hosts both the backend and the managed database, ensuring that the system can be deployed, demonstrated, and tested consistently without relying on local configuration. The architecture prioritises correctness, reliability, and clear separation of concerns.

3.6.6 Architectural Trade-offs

A few design decisions are worth explaining explicitly.

Database triggers vs. application-level validation. Oversell prevention lives in a PostgreSQL trigger and check constraint, not the application. Two concurrent requests can't both succeed if only one slot remains – the database rejects one at the write level. The downside is that trigger errors are harder to debug than application exceptions. Worth it for a marketplace where overselling is fatal.

Stateless JWT vs. session-based auth. JWTs keep the backend stateless, which suits Railway's deployment model. The trade-off is no server-side revocation – a compromised token stays valid until it expires. Fine for this scope; a production system would add a token blocklist.

Monorepo vs. separate repos. One repo simplified CI and deployment handover. The cost is coupling: a failing frontend lint check blocks everything. For an 8-person coursework team, that trade was worth it.

Spring Boot vs. alternatives. The whole team knew Java, and Spring's structure kept contributions consistent across 8 developers. It's heavier than Express and slower to start, but team familiarity mattered more over a short sprint.

4. Forecasting

A summary of the forecasting system in the final version of Byte Me. Covers the three models, evaluation methodology, and known limitations.

4.1 Models Implemented

Three models are implemented and compared on held-out data.

Moving Average (Baseline 1). Mean of the last 4 weeks for the same seller. Simple, stable, no trend or day-of-week adjustment. Confidence range: 0.3–0.7.

Seasonal Naive (Baseline 2). Repeats the value from the same day-of-week in the prior week. Captures weekly seasonality but volatile on sparse data. Confidence range: 0.35–0.5.

EMA-Weighted (Selected). EMA smoothing ($\alpha = 0.3$) with four adjustment factors: day-of-week (1.0–1.4×), weather (boolean flag), discount effect (0.2 multiplier), and trend (0.8–1.3×). Confidence range: 0.4–0.85.

Model	**How it works**	**Confidence range**
Moving Average (Baseline 1)	Mean of last 4 weeks	0.3–0.7
Seasonal Naive (Baseline 2)	Same day-of-week from prior week	0.35–0.5
EMA-Weighted (Selected)	EMA (α=0.3) + day-of-week (1.0–1.4×) + weather + discount (0.2×) + trend (0.8–1.3×)	0.4–0.85

Table 4.1: Forecasting models implemented in the final system.

4.2 Evaluation Methodology

All models use an 80/20 held-out split – earliest 80% trains, latest 20% evaluates. Three metrics: MAE for count accuracy, RMSE to penalise large errors, Brier Score for no-show probability calibration. Best model is selected by lowest MAE. Results are persisted in `ForecastRun` and accessible via `/api/forecast/evaluate/{sellerId}`.

4.3 Why EMA

The moving average can't respond to recent trends. The seasonal naive model repeats whatever happened last week – if that was an anomaly, the prediction is wrong. EMA dampens outliers while still weighting recent data, and the day-of-week and discount factors capture structure neither baseline can represent. EMA achieves lower MAE than both baselines on the seeded dataset.

4.4 Limitations

The weather input is a boolean flag, not a live API. The adjustment is applied statically rather than in response to real conditions. The dataset is also seeded, not real – accuracy claims are

indicative only. A production deployment would need several weeks of real data before the evaluation results mean much.

5. Success Criteria

This chapter outlines the key measures used to evaluate whether the Byte Me platform achieves its intended goals. The criteria focus on reliability, forecasting performance, environmental impact, usability, and user engagement. Each measure is grounded in implemented system behaviour rather than theoretical design.

5.1 Marketplace Reliability

Reliability is the most fundamental success measure. A food rescue platform must be trusted by sellers and organisations, which means the system must reflect real-world availability accurately at all times.

5.1.1 Oversell Prevention

The platform is designed so that it is impossible for more reservations to be made than the quantity originally posted. Before a reservation is confirmed, the system verifies that the requested quantity does not exceed the remaining available stock and that the bundle is still active. If this condition is not satisfied, the reservation request is rejected immediately. Because this validation is enforced transactionally, overselling cannot occur even under simultaneous requests. As a result, the oversell rate is strictly zero by design rather than by assumption. This behaviour is directly testable and provides a measurable guarantee of marketplace integrity.

Result: Passed. The `reservation_capacity_on_insert` trigger and `chk_qty_reserved_le_total` constraint enforce this at the database layer. `OrderControllerTest` covers concurrent scenarios and manual test case TC-03 tests race conditions directly.

5.1.2 Handling Expiries and No-Shows

In practice, not every reserved bundle is successfully collected. The system therefore distinguishes clearly between successful collections, cancellations, no-shows, and expired listings. Bundles that pass their pickup window without being collected are automatically identified and marked appropriately. This prevents outdated listings from remaining visible and ensures that inventory reflects real availability. By explicitly modelling each possible outcome in the bundle lifecycle, the platform maintains transparency and traceability for both sellers and organisations.

Result: Passed. Status transitions (RESERVED → COLLECTED / CANCELLED / EXPIRED) are implemented and covered in `OrderControllerTest`.

5.2 Forecast Accuracy

Beyond core reliability, the platform aims to help sellers make better posting decisions using demand forecasting. Forecast performance is evaluated using a held-out validation approach, where historical data is split into training and evaluation sets. The chosen forecasting model is then compared against simpler baseline approaches. Three error metrics are used:

- Mean Absolute Error (MAE), measuring average prediction deviation.
- Root Mean Squared Error (RMSE), which penalises larger errors more heavily.
- Brier Score, evaluating the accuracy of no-show probability predictions.

By comparing the selected model against a moving average baseline and a seasonal naive benchmark, the system ensures that any improvement in predictive performance is evidence-based.

Forecast results are stored and monitored over time, allowing continuous evaluation rather than one-off validation.

Result: Passed. EMA outperforms both baselines on MAE. Pipeline covered by `ForecastServiceTest`; results accessible at `/api/forecast/evaluate/{sellerId}`.

5.3 Waste Impact and Carbon Reduction

The environmental objective of Byte Me is measurable reduction in food waste. For each successfully collected bundle, the system calculates the estimated weight of food diverted from landfill. This is derived from the seller's estimated bundle weight and the number of completed collections. Using established food waste conversion factors, the platform can also estimate associated CO_2e savings. The CO_2e factor used is 2.5 kg per kg of food waste, sourced from WRAP UK data. Category-specific seed weights are applied where available (bakery 2500g, produce 4000g, dairy 3500g, hot meals 6000g, lunch 5000g), with a default of 1500g per bundle otherwise. These impact metrics are aggregated weekly and displayed on the seller dashboard, providing tangible evidence of environmental contribution.

Result: Passed. `/api/analytics/waste-avoided/{sellerId}` is live. Waste Avoided Estimation Report included in the handover pack.

5.4 User Experience

A rescue platform must operate efficiently in retail environments where speed matters. Reservation confirmation and collection verification are designed to occur instantly. Claim codes are generated at the time of reservation and securely validated during pickup. The process is intentionally simple for staff while maintaining appropriate security safeguards. The success measure here is responsiveness and clarity: users should be able to reserve and complete collections without friction or delay.

Result: Passed. 191 frontend tests cover loading states, error handling, form validation, and access control. Claim code flow tested manually end-to-end.

5.5 Engagement and Gamification

Long-term impact depends on continued participation. The system therefore tracks organisational rescue streaks and awards milestone badges based on consistent activity. Streak updates and badge eligibility checks occur automatically after successful collections. This provides immediate positive reinforcement and encourages repeated engagement. By linking measurable environmental impact with visible milestones, the platform supports sustained user motivation.

Five badges: FIRST_RESCUE (1 collection), STREAK_4 (4-week streak), CO2_SAVER (10 collections), VARIETY_SELLERS (3+ distinct sellers), VARIETY_CATEGORIES (3+ distinct categories). The streak system is ISO week-based – increments on consecutive weeks, resets on gaps, tracks current and best.

Result: Passed. Streak and badge logic covered by `GamificationController` tests. Updates trigger automatically on collection confirmation.

Summary in Plain Terms

In simple terms, the system is considered successful if it never oversells food, keeps listings accurate, predicts demand better than basic averages, clearly measures how much waste is saved, works quickly for users, and encourages organisations to keep participating. Each of these goals

is supported by implemented logic and measurable data, meaning the platform's performance can be evaluated objectively rather than assumed.

6. Evaluation and Critical Reflection

How the final system was tested, how it performed against its own success criteria, and an honest account of the remaining limitations.

6.1 Evaluation Method

Testing ran across four layers. Backend: 28 test files, 142 methods, 3224 lines. Frontend: 13 files, 220 tests covering loading states, error conditions, form validation, and role-based access control. Two GitHub Actions CI pipelines run on every push – backend tests and frontend checks. Manual testing covered five documented test cases: core reservation flow, concurrent oversell prevention, claim code verification, streak update logic, and issue reporting. A UAT video walkthrough was recorded as part of the handover pack.

Forecasting evaluation uses the held-out methodology described in Chapter 4, reproducible via the `/api/forecast/evaluate/{sellerId}` endpoint.

6.2 Results Against Success Measures

5.1 Marketplace Reliability – Passed. Zero oversell rate by design. Concurrent scenarios covered by automated tests. Status transitions fully implemented and tested.

5.2 Forecast Accuracy – Passed. Three models evaluated on held-out data. EMA wins on MAE. Covered by `ForecastServiceTest`.

5.3 Waste Impact – Passed. CO_2e calculation uses WRAP UK factor of 2.5 kg/kg. Analytics endpoint is live. Methodology documented in handover pack.

5.4 User Experience – Passed. 191 frontend tests confirm correct behaviour. Claim code flow tested manually end-to-end.

5.5 Engagement and Gamification – Passed. Five badges and ISO week-based streaks implemented and tested. Triggers automatically on collection confirmation.

6.3 Limitations

The weather input is a boolean flag, not a live API. The issue reporting UI is a placeholder – the backend works but the frontend page was cut. WCAG 2.2 AA conformance is partial and self-evaluated only. No rate limiting on auth endpoints. No token refresh mechanism – users re-login after 24 hours. The evaluation dataset is seeded, not real traffic. Map view was cut deliberately in favour of forecasting depth.

6.4 Societal and Environmental Impact

CO_2e savings are calculated per collection and surfaced to sellers, so the environmental impact is auditable rather than just claimed. All 14 UK allergens under Natasha's Law are listed on bundle detail pages with popup reminders on the reservations page.

There is no data deletion mechanism – a GDPR gap that would need addressing before any public launch. Gamification uses positive reinforcement only; gambling-style mechanics were explicitly excluded in the MoSCoW and aren't in the final system. The B2B model targets bulk rescue at organisational scale, which is more likely to sustain high-volume impact than individual consumer app usage.

7. What Changed Since CW1

A summary of the structural and functional changes made during Sprint 2.

7.1 System Growth

The API went from 20 endpoints to 41. Analytics went from two views to six. Forecasting went from one EMA model to a three-model comparison framework with automated selection by MAE.

7.2 Structural Changes

The V2 Flyway migration added `forecast_action` for tracking seller responses to recommendations. Four new entities were introduced: `DemandObservation`, `ForecastRun`, `ForecastOutput`, `SellerMetricsWeekly`. These are reflected in the updated ER diagram in Chapter 3.

7.3 New Features Delivered in Sprint 2

- Issue reporting – six endpoints, OPEN → RESPONDED → RESOLVED state machine.
- Full analytics suite – six seller dashboard views.
- Three-model forecasting comparison with MAE/RMSE/Brier and confidence intervals.
- Accessibility improvements – read-aloud, contrast fixes, keyboard navigation.
- Terms of Service, Privacy Policy, and Cookies Policy pages live.
- Setup scripts for macOS and Windows.
- Client handover pack – deployment guide, environment variables, migration steps, admin credentials.

7.4 Test Coverage Growth

Metric	**Value**
Backend test files	28
Backend test methods	142
Frontend test files	13
Frontend tests	∼220
API endpoints	41
Database triggers	6
Database check constraints	7+
Flyway migrations	2 versioned + 1 repeatable seed
Frontend routes	24
Badges	5
Forecasting models	3
Forecasting metrics	3 (MAE, RMSE, Brier)
CI workflows	2 (backend tests, frontend checks)
CO_2e factor	2.5 kg per kg waste (WRAP UK)
Default bundle weight	1500g
JWT expiry	24 hours
EMA alpha	0.3

Table 7.1: System metrics for the final delivered system.

Bibliography

[1] FAO, F. A. A. O. O. T. U. N. FAO Policy Series: Food Loss & Food Waste, 2013.

[2] Ltd, I. D. Survey Reveals Employee Productivity Averages 2 Hours and 53 Minutes a Day, Jan. 2025.

[3] WRAP. UK Food Waste & Food Surplus - Key Facts. Tech. rep., July 2025.